CHAPTER 4: DESIGN AND IMPLEMENTATION GUIDELINES

A practical approach for BSIT and BECE programmes

Prepared by Dr M Chinyuku, 2026

4.0 Introduction

The chapter should cover the design of output, input, database/file, and security control specifications. Justify the programming language chosen. Outline the functional and system requirements. Include the system architecture; you may also incorporate DFDs, flowcharts, use cases, and sequence diagrams, depending on the nature of your project.

Begin by establishing:

- The **type of project** (Software, Algorithm/Simulation, Framework, or Hardware)
- The **system objective**
- A brief recap of the **research gap being addressed** and how this chapter connects to the **research gap identified in Chapter 2**
- The **approach adopted to solve the problem**

Also, outline what this chapter will cover:

- System requirements
- Architecture and design
- Implementation strategy
- Evaluation approach

Example

This chapter presents the design and implementation of the system. It outlines the system requirements, architectural design, implementation plan, and the tools and technologies used. The chapter also describes how the solution addresses the problem identified in Chapter 2.

The project is a **[Software / Algorithm / Framework / Hardware]** based system designed to **[state objective clearly]**.

4.2 System Architectural Design

Describe the **high-level system structure**:

- Overall system structure (e.g., layered, client-server, embedded system, microservices, MVC, etc.)
- Major components/modules and their interactions

Include:

- System Architecture diagram
- Module decomposition
- Justification of design choice and
- Explain how it satisfies system requirements

Avoid decorative diagrams. Every box and arrow must mean something. If you can remove a component and nothing changes, it shouldn't be there.

4.3 System Requirements and Specifications

4.3.1 Functional Requirements

Describe what the system must do:

- Core operations (e.g., data input, data processing, prediction/classification, output, user authentication)
- User interactions

Example:

The functional requirements describe what the system is expected to do.

- FR1: [Describe function]
- FR2: [Describe function]
- FR3: [Describe function]

4.3.2 Non-Functional Requirements

Define quality attributes:

- Performance (e.g., response time, throughput)
- Security
- Usability
- Reliability
- Scalability

Make these measurable. Engineering without measurable constraints is philosophy wearing a lab coat.

Example:

Bad: “System should be fast.”
Good: “System response time < 2 seconds under 100 concurrent users.”

Example:

The non-functional requirements define system quality attributes.

- Performance: [e.g., response time < 2 seconds]
- Security: [e.g., user authentication required]
- Usability: [e.g., user-friendly interface]
- Reliability: [e.g., system uptime requirement]
- Scalability: [if applicable]

4.3.3 Input, Output, Database/File and Security Design

Input Design

Describe all system inputs:

- Data sources (user input, sensors, files, APIs)
- Input validation mechanisms

Example:

- Input sources: [user input, sensors, files, APIs]
- Input format: [text, numeric, images, etc.]
- Validation rules: [e.g., range checks, format validation]

Output Design

Describe system outputs:

- Reports, dashboards, alerts
- Format and presentation of results

Example:

- Output types: [reports, predictions, alerts]
- Format: [tables, graphs, dashboards]
- Output destination: [screen, file, device]

Database/File Design

Describe how data is stored and managed:

- Data structures and storage mechanisms
- Tables/files and relationships (linked to ERD)

Example:

- Database type: [Relational / NoSQL / Files]
- Tables/Entities:
    - Table 1: [Name + description]
    - Table 2: [Name + description]
- Relationships (refer to ERD)

Security Control Specifications

Describe system security mechanisms:

- Authentication and authorisation
- Data protection mechanisms
- Error handling and system integrity

Example:

- Authentication: [login system, credentials]
- Authorisation: [user roles]
- Data protection: [encryption, validation]
- Error handling and system integrity measures

A system that produces correct results but leaks data is not “partially correct”—it is fundamentally broken.

4.3.4 System Modelling and Design Tools

Students must **correctly apply and interpret** the tools. Use only what is relevant to your project:

a) **Flowcharts**
   - Explain Process logic
   - Show step-by-step logic of processes
   - Use correct symbols (terminator, process, decision)

b) **Data Flow Diagrams (DFDs)**
   - Explain data movement
   - Show data movement between processes, data stores, and external entities
   - Include:
     - Context diagram (Level 0)
     - At least one Level 1 decomposition

c) **UML/Class Diagrams** – System structure
   - Use Case Diagram -Explain actors and interactions
   - Sequence Diagram-Explain interactions over time
   - Class Diagram (object structure)

d) **ERDs** – Database design
   - Define and explain database structure:
     - Entities
     - Attributes
     - Relationships
     - Cardinality

e) **Structure Charts / HIPO** – Module hierarchy
   - Show module hierarchy and data/control flow

f) **Decision Tables** – Represent Complex decision logic

g) **Data Dictionary** – Definition of all data elements

- Define all data elements:
  - Name
  - Type
  - Description
  - Source

Each diagram must:

1. Be explained
2. Be captioned
3. Be consistent with others
4. Map directly to implementation

Only include diagrams that you understand

4.4 Project Category

4.4.1 Software Solution

- Detailed module design
- Interface design
- Integration of components

4.4.2 Algorithm / Simulation

Describe:

- Structure of the algorithm or simulation system
- Algorithm architecture (input → process → output pipeline
- Tools/libraries used (e.g., Python, MATLAB)

Pseudocode

Provide:

- Clear, structured pseudocode
- Language-independent
- Logical steps only (no syntax noise)

Pseudocode should read like structured thinking, not broken programming.
Example:

```
BEGIN
  Step 1: …
  Step 2: …
END
```

Flowcharts

- Visual representation of algorithm logic
- Must align with pseudocode

Algorithm Analysis:

Include:

- Time complexity (Big-O notation)[e.g., O(n log n)]
- Space complexity[e.g., O(n)]
- Performance metrics (accuracy, efficiency)

Big-O notation $O(n^2)$ is not decoration; it is a claim about how your system behaves on scalability. (Do not use it casually)

If ML-based:

- Training time vs inference time
- Accuracy, precision, recall

4.4.3 Framework / Model-Based Design

Describe

- Framework architecture/structure
- Data flow within the model
- Integration of subsystems

Data Analysis:

Include:

- Data sources
- Data preprocessing steps
- Feature selection
- Visualisation (if applicable)

Raw data is messy. Explain how you organised it before using it in your model.

Theoretical Model:

Explain:

- The theory underpinning the framework/Model used
  (e.g., Machine Learning model, optimization model, statistical model, neural network, regression etc)

Include:

- Equations/principles (where applicable)
- Model assumptions
- Justification of model choice

Remember: A model built on no assumptions is destined to fail

4.4.4 Hardware Design

- System architecture (block diagrams)
- Board/circuit design
- Component (microcontroller, sensors, actuators etc) selection and justification
- Hardware interface design (user interaction)

Board Design

Explain:

- Circuit design
- Components used and why
- PCB layout (if applicable)

Include:

- Circuit diagrams
- Pin configurations

Describe:

- User Interface Design – describe how users interact with the system/hardware
- Input/output mechanisms:
    - Buttons
    - Displays
    - Indicators

Good hardware design is invisible—the user shouldn't need a manual to press a button.

4.5 Implementation

Describe the actual system development:

4.5.1 Implementation Overview

Describe:

- The approach to implementation
- Tools and technologies used
    - Programming languages [e.g., Python, Java, C++]
    - Development environment
    - Libraries/frameworks
- Integration of components (process)

Explain:

- Why this language/tool was selected
- Suitability for the problem (performance, libraries, ecosystem)

Include:

- Screenshots (software systems)
- Prototype images (hardware systems)

4.5.2 Deliverables

List and describe all deliverables:

- Executable files
- Source code
- Data files
- Documentation
- User manual
- Sample outputs/results

4.5.3 Milestone Identification

Define key project phases:

- System analysis
- System design
- Database/file design
- Module design
- Coding
- Testing
- Documentation

4.5.4 Milestone Completion Criteria

Specify how each milestone is evaluated:

- Objective measures (e.g., “module compiles without errors”)
- Subjective evaluation (e.g., supervisor approval)

4.5.5 Schedule of Milestone Completion

Present:

- Timeline (table or Gantt chart)
- Expected completion dates

4.6 Chapter Summary

Purpose of the Summary

The summary should:

- Recap the **key design and implementation elements**
- Reinforce how the system **addresses the research problem**
- Highlight **important decisions and outcomes**
- Prepare the reader for **Chapter 5**

Example:

This chapter presented the design and implementation of the system. It outlined the functional and non-functional requirements, as well as the input, output, database, and security specifications necessary for system development.

The system architecture was described, highlighting the key components and their interactions. Various modelling tools, including [mention diagrams used, e.g., use case diagrams, DFDs, flowcharts], were utilized to represent the system design.

The implementation approach was also discussed, including the tools and technologies used. The choice of programming language, **[insert language]**, was justified based on its suitability for the system requirements.

Furthermore, the chapter detailed the [algorithm/framework/hardware] design, including a brief mention of key elements such as pseudocode, model, or circuit design. The implementation plan was presented, outlining deliverables, milestones, and the project schedule.

Overall, the chapter demonstrated how the system was designed and implemented to address the identified problem.

The next chapter presents the testing and evaluation of the system to assess its performance and effectiveness.

4.7 General Comments

4.7.1 Consistency is Everything

Requirements → Design → Implementation must align perfectly.

If your design shows 5 modules but your implementation has 2, something went wrong—or something was imagined.

4.7.2 Avoid Over-Engineering

Do not include *every diagram you have ever learned*. Use only what adds clarity.

4.7.3 Justify Every Decision

Every choice should answer:

- Why this approach?
- Why not alternatives?
- Why this algorithm?
- Why this architecture?
- Why this tool?

4.7.4 Think Like an Engineer, Not a Narrator

A narrator describes what happened.
An engineer explains:

- Why it works
- Under what conditions it fails
- How it can scale

4.7.5 Overpromising vs Implementation Reality

Students often describe:

"AI-powered intelligent system"

Then implement:

A basic if-else rule system

That mismatch is the fastest way to lose marks.

4.7.6 Diagram–Implementation Disconnect

Every diagram must map to:

- Code
- Algorithm
- Hardware

4.7.7 Missing Integration

Students design parts but never show:

- How everything works together

A system is not a collection of parts—it's an interaction of parts.